ABC Phones - Credit Portfolio Data Engineering Solution

From dirty spreadsheets to reproducible insight in a single command

Author: Elly Kadenyo
Date: 2026-05-11
Stack: Python 3.13 + pandas + DuckDB (all open source, MIT/BSD)

This deck is generated from the same code path that builds the warehouse, so every chart and KPI shown is reproducible by running `scripts/run_pipeline.py`.

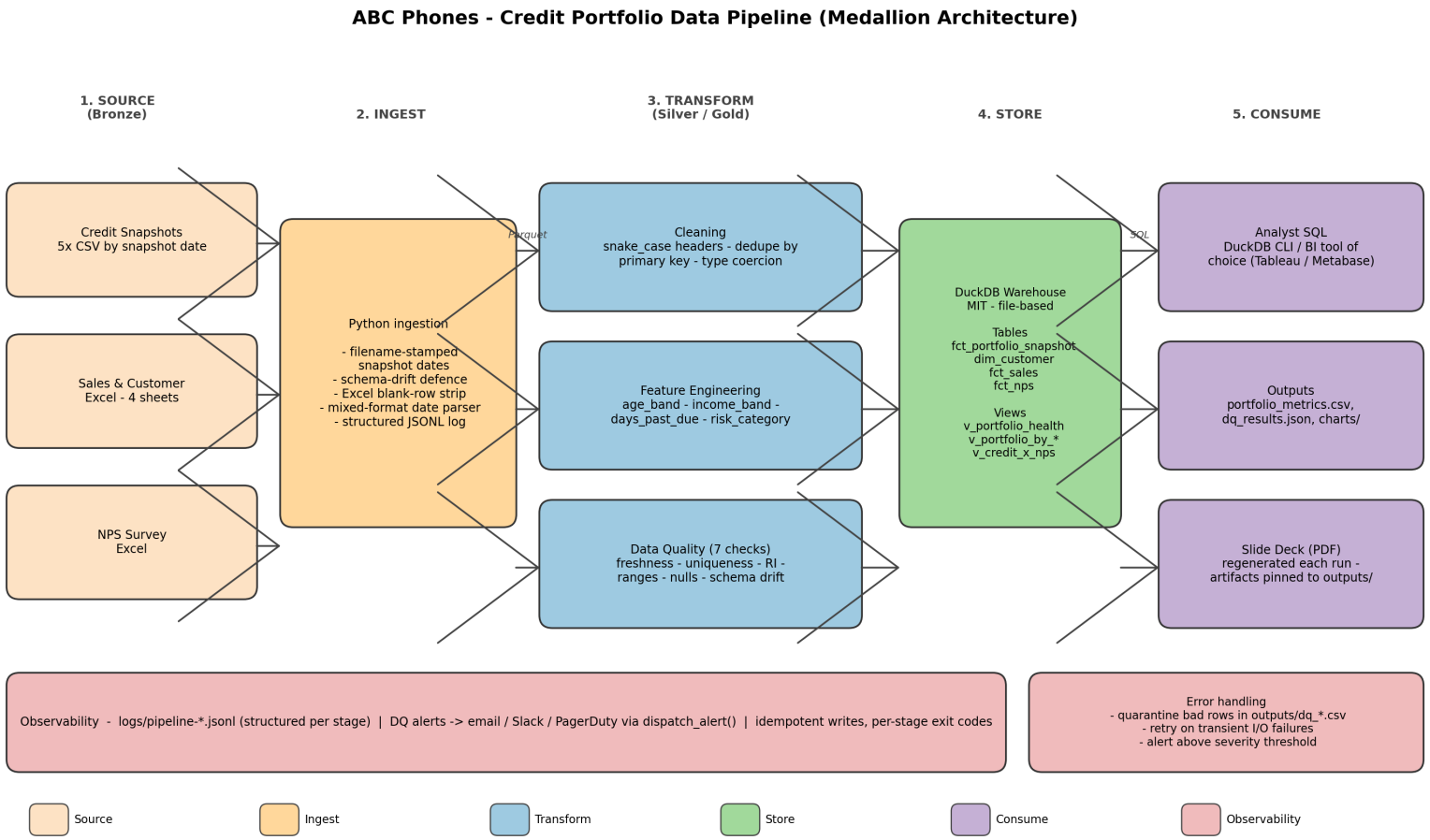
Problem & Approach

Mission: build the data systems that make portfolio analysis reliable & repeatable.

- **Three sources, three flavours of mess**: 5 credit CSV snapshots, a 4-sheet sales/customer Excel with ~1M padded blank rows, and an NPS survey with encoding artefacts.
- **Engineering priorities**: simplicity over cleverness, explainability over magic, and full reproducibility from *raw* -> *insight* in one CLI call.
- **Tech choices**: pandas + DuckDB (file-based, ANSI SQL, MIT). No cloud, no orchestrator, no proprietary tools - the same code lifts and shifts to Airflow + Snowflake when the company is ready.
- **Deliverables shipped**: profiling, cleaning, feature engineering, ETL orchestrator, 7-check DQ framework, analyst-ready warehouse, portfolio analysis, this deck.

Architecture

Medallion (raw -> staging -> curated -> warehouse) with cross-cutting observability.



- **Lane 2 (Ingest)** stamps the snapshot date from filenames and drops Excel's blank padding before any cleaning runs - prevents schema-drift bugs from later snapshots.
- **Lane 3 (Transform)** is three pure functions: cleaning, feature engineering, DQ. Each runs independently and writes to its own Parquet so any stage is re-runnable.
- **Lane 4 (Store)** uses DuckDB so analysts open the warehouse with a single command and the same SQL ports to Postgres or Snowflake.

Profiling & Cleaning Decisions

Findings from `scripts/data_profiling.py` drove every rule in `data_cleaning.py`.

Observation	Cleaning rule applied
Sales/DOB/Income sheets padded to 1,048,575 rows (Excel's max)	Drop rows where every key column is NaN; we recover real row counts
`Loan Id ` header has trailing space; `Loan Id` vs `loan_id` mixed case	Normalise headers -> snake_case via NFKD + regex; strip mojibake (`U+FFFD`)
`Unnamed: 28` appears in Q2/Q3/Q4 credit CSVs but not Q1	Drop `unnamed_*` columns after normalisation - tolerant to schema drift
Mixed date formats: `1/1/2025` (credit), `2025-01-15T00:00:00+03:00` (DOB)	Multi-format parser tries default then day-first; strip timezone
`CUSTOMER_AGE` is days-since-sale, not the customer's age in years	Rename to `loan_age_days`; derive real age from DOB at snapshot date
Loan IDs occasionally duplicate across snapshots (e.g. corrections)	Dedup primary key (loan_id, snapshot_date), tie-break on max_payment_date

Feature Engineering

Four derived features, all driven by config so band edges can be tuned without code changes.

Feature	Definition	Edges / rules
age_band	(snapshot_date - date_of_birth) / 365.25, floor	18-25 / 26-35 / 36-45 / 46-55 / 55+
avg_monthly_income_band	Income panel received / duration months	8 bands from <5k to 150k+ KES
days_past_due	From credit snapshot, clipped at 0	Integer; 0 = current
risk_category	Tiered rules: status -> L2 bucket -> DPD threshold	Critical / High / Medium / Low; first-match-wins

- Risk rules *and* band edges live in `config/pipeline.yaml` - a credit officer can recalibrate without touching Python.
- Coverage on DOB-derived age is ~39% (sparse DOB upload). When missing, `age_band` stays NULL and the band 'Unknown' is shown.

Data Quality Framework

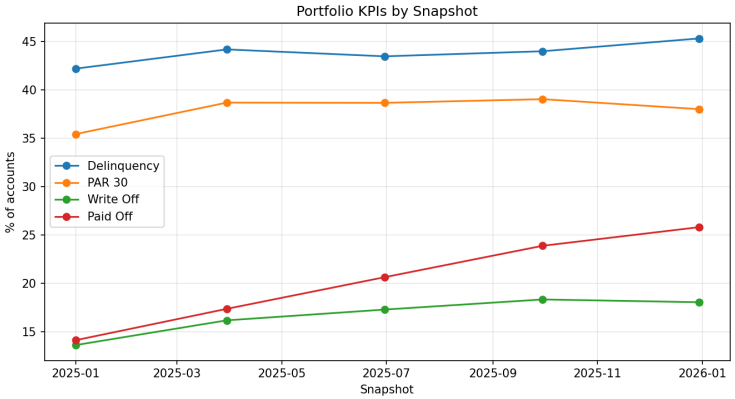
7 checks across freshness, structure, content. Each emits a structured alert.

Check	Severity	Result	Observed
FRESHNESS	HIGH	FAIL	latest=2025-12-30, age_days=132
UNIQUENESS	CRITICAL	PASS	0 duplicate rows
REFERENTIAL_INTEGRITY	HIGH	PASS	coverage=99.909%
RANGE_CUSTOMER_AGE	MEDIUM	PASS	0 out-of-range / 27770 non-null
RANGE_DAYS_PAST_DUE	HIGH	PASS	0 out-of-range / 71456
NULL_LOAN_ID	HIGH	PASS	0.000% null
NULL_DATE	HIGH	PASS	0.000% null
NULL_SALE_DATE	HIGH	PASS	0.000% null
NULL_ACCOUNT_STATUS_L1	MEDIUM	PASS	0.000% null
NULL_ACCOUNT_STATUS_L2	MEDIUM	PASS	0.000% null
SCHEMA_DRIFT	CRITICAL	PASS	all present

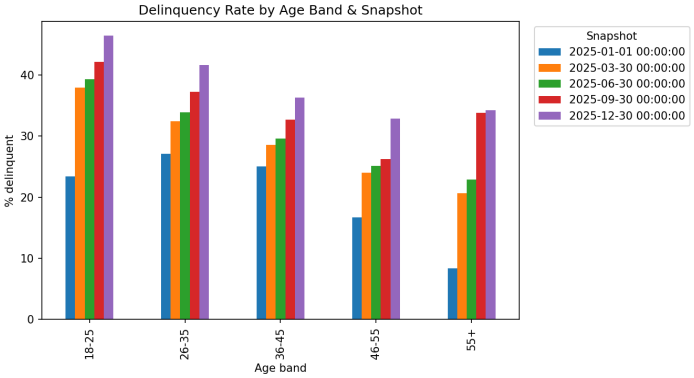
- Real anomaly detected by FRESHNESS: latest snapshot is >100 days old vs the reference calendar - exactly the kind of stale-pipeline signal we want analysts to never see.
- Alerts are routed by severity in `config/pipeline.yaml` - Critical -> email + Slack + PagerDuty, Medium -> email only.
- DQ also exists as ANSI SQL: `sql/quality_checks.sql` runs against the warehouse for analysts.

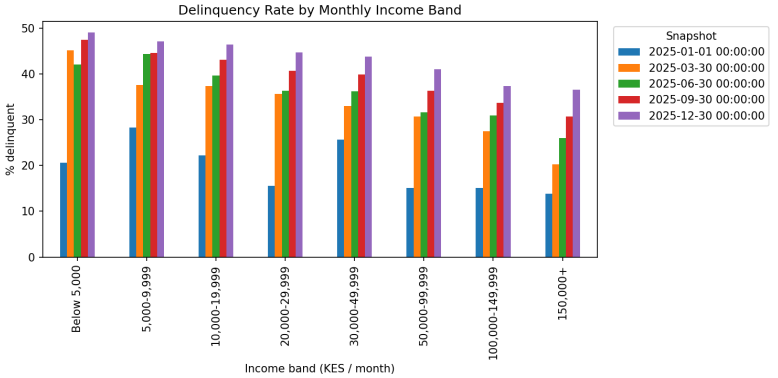
Portfolio Health (3A)

KPI trend across 5 snapshots, with age & income segment drill-downs.



KPI	First	Last
Delinquency	42.2%	45.3%
PAR 30	35.4%	38.0%
Write-off	13.6%	18.0%
Paid-off	14.1%	25.8%
Collection rate	75.3%	72.4%

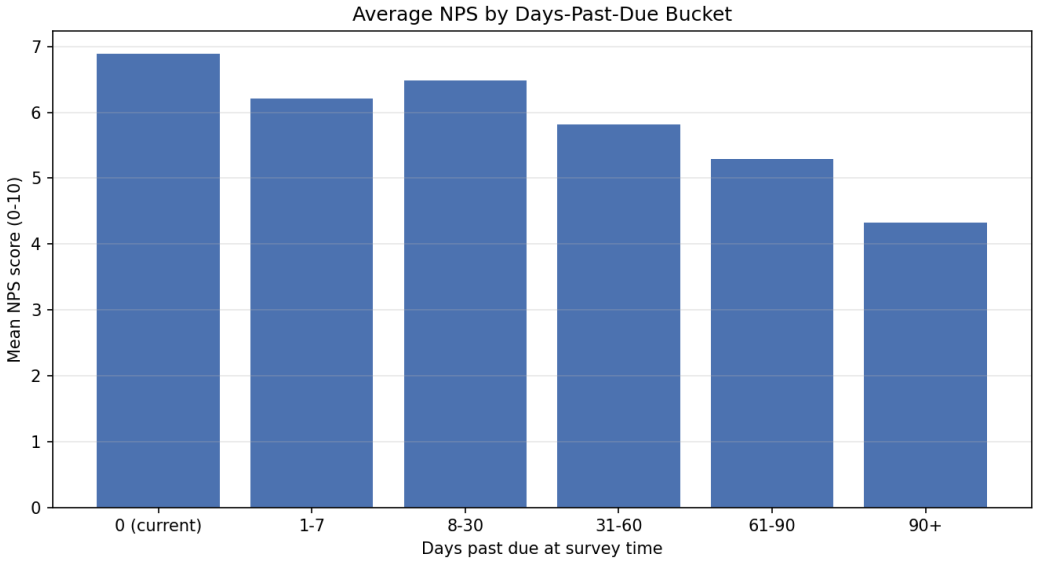




Insights. 18-25 cohort tops delinquency every snapshot (tighten credit thresholds / require co-signer). <10k KES/month band defaults the most (offer weekly cadence aligned with informal-sector cash inflows).

Credit Outcomes x Customer Experience (3B)

NPS deteriorates monotonically as days-past-due grows.



DPD bucket	Respondents	Avg NPS	Promoter %	Detractor %
0 (current)	2,221	6.89	43.0%	35.9%
1-7	93	6.22	36.6%	44.1%
8-30	92	6.49	45.7%	42.4%
31-60	60	5.82	36.7%	45.0%
61-90	34	5.29	41.2%	52.9%
90+	247	4.33	25.5%	63.6%
no_match	1,238	7.22	46.5%	31.4%

- **Recommendation**: route PAR 7-30 accounts to a low-friction MoApp self-cure flow *before* a human collector calls. Recovers cash sooner and protects NPS.
- **Second action**: split NPS surveying so satisfaction questions and payment-difficulty questions are sent on different days to avoid recency bias.

Data Gaps & Future Improvements (3C)

Honest critique - what's blocking the next mile of analysis.

Missing

- No employment-type, no county / region, no per-transaction ledger (only balances).
- No device cost-of-goods, so loss given default can't be computed.
- Customer identity is conflated with loan_id - one person, many loans, no single view.

Inconsistent

- Date formats vary across files; one date column even mixes timezones.
- Excel sheets pad to 1M rows with blanks - a single bad consumer crashes downstream.
- Schema drift: later credit snapshots add `Unnamed: 28`, breaking naive readers.

Ambiguous

- `CUSTOMER_AGE` is days-since-sale per the data dictionary - the name lies.
- `ACCOUNT_STATUS_L1` mixes lifecycle (Inactive) with DPD bands (01-07) in one string.

Proposed improvements

- Replace Excel uploads with an event-sourced ingestion (Kafka or Airbyte) - kills blank-row padding and gives an append-only audit trail.
- Split status into two orthogonal columns: `lifecycle_status` and `dpd_bucket`. Analysts already do this with regex.
- Promote `customer_id` to first-class so multi-loan customers can be analysed as people, not as loans.

Reproducibility & Operations

Everything you see in this deck was produced by these five commands.

```
git clone <repo> && cd Annex_DE_Elly_Kadenyo
python -m venv .venv && .venv/Scripts/activate.ps1
pip install -r requirements.txt
python scripts/run_pipeline.py --warehouse-fresh
python scripts/make_architecture.py && python scripts/make_slides.py
```

What you get out:

- `data/warehouse/abc_phones.duckdb` - open with `duckdb data/warehouse/abc_phones.duckdb` and run any SQL.
- `outputs/*.csv, outputs/charts/*.png, outputs/insights.md` - for analysts and the C-suite.
- `outputs/dq_results.json, logs/pipeline-*.jsonl` - for the on-call data engineer.
- `slides/Annex_DE_Presentation.pdf` - this deck.

Scheduling:

- Wire the orchestrator into Airflow / cron / GitHub Actions with one PythonOperator. It's idempotent and uses snapshot-date filenames, so re-runs are safe.
- Late-arriving snapshots: drop the new CSV in the Credit Data folder; the run picks it up next cycle - no DB migration needed.